

23A – M3 Pricing & Rules: Implementation Guide

Everything the M3 developer and the till track need: the pricing schema as DDL, seed data, package layout, the API slice, handler specifications, the shared engine in Kotlin (price resolution, rule evaluation, tax and rounding), the authoring validator, the review consumers, the snapshot contributor, golden-basket fixtures, screens, tests and tickets. Self-contained; restated material is marked.

Field	Value
Document	23A – implementation guide for 23 M3 Pricing & Rules v0.1
Version	v0.1 – for the M3 developer and the till track (Section 6 is shared)
Date	9 September 2026
Prerequisite	Sprint 0; M1 and M2 contracts frozen; the shared-engine module of 17A §9 with its interfaces. M5's in-stock batch query is stubbed until doc 25 (Section 11).
Estimated effort	About 22 developer-days: 15 for the module, 7 for the engine (of which the till track co-owns the parity suite)
Restated content	Marked <i>restated from doc X §Y</i> .

Read this first. Build the engine before the module: it is a pure Kotlin library with no database, it is what the till runs, and every back-office screen just calls it. Get the golden baskets green on JVM and Android first (ticket M3-02), then wrap the tables and screens around it. The legal property – no output ever exceeds a control price – is a property-based test, not a code review item.

1. What you are building

Restated from doc 23 §1. Every price a buyer pays and every reduction a customer receives: trade lists per relationship, one retail list per MPCS, advisory prices, a five-kind rule engine as the only reduction mechanism, the printed-MRP and gazetted ceilings as hard blocks, the multi-MRP policy, and the resolution engine shared with the till.

You build	You do not build
Schema pricing: price_list, price_list_line, discount_rule, control_price, mrp_policy, adjustment_type, label_job	Batch MRP itself (M2), lots (M5), documents that carry prices (M4, M6)
The shared engine's real implementation: PriceResolver, RuleEvaluator, Tax, Rounding, BasketResolver	The till application (M6) – it links your library
11 commands, 10 queries, 6 event families; the authoring validator; two review consumers; the snapshot contributor; the label job	Weigh labels and batch stickers (M6, M5)
Seven web screens; the MRP picker's data contract for M6	The picker screen (M6)

2. What you inherit

From	You use	How
17A §9 shared-engine	Module skeleton: Money, Quantity, Tax, PriceResolver, RuleEvaluator, Envelope interfaces with stubs	You replace the stubs (Section 6); keep the package and versioning
19A §4–5, §11–12	AuditFacade, EventPublisher, ConfigRegistry, @ScheduledJob, @EventConsumer	As in every module
19A §6 i18n	A4Renderer for labels; lcuFormats for money	Label job
19A §8 sync	SnapshotContributor; SnapshotChangeLog.record(..., applyFrom, urgent)	Section 7.3
M1 api/query	LookupRelationship (trade list binding), listLocations(entity), scope	Trade resolution; change-log fan-out per location
M2 api/query	GetSku (flags, has_printed_mrp, tags), TaxQueries.rateInForce, TagQueries.isGoverned, batch identity	Authoring checks; tax; policy scopes
M5 query (stub until doc 25)	InStockBatches(locationIds, skuld) → [(batchId, printedMrp, expiry, onHand)]	Authoring ceiling check (min in-stock MRP)
17A §7 web shell	DocumentHeader, ApprovalBar, ReasonCapture, StepUpDialog, generated client	Section 8

3. Schema

Restated from doc 18 v0.2 part B (M3 tables) and doc 23 §3; adds mrp_policy, adjustment_type, label_job (CR-23-1). Location db/migration/m3pricing/.

```
-- V0001_pricing.sql
CREATE TABLE pricing.price_list (
  price_list_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL,
  kind text NOT NULL CHECK (kind IN ('TRADE','RETAIL','ADVISORY')), name text NOT NULL,
  version int NOT NULL DEFAULT 1, source_version_id uuid,
  status text NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','PUBLISHED','SUPERSEDED','WITHDRAWN')),
  apply_from date, published_by uuid, published_at timestamptz;
-- one PUBLISHED RETAIL list per MPCs
CREATE UNIQUE INDEX one_published_retail ON pricing.price_list (owner_entity_id) WHERE kind = 'RETAIL' AND
  status = 'PUBLISHED';
CREATE UNIQUE INDEX one_draft_retail ON pricing.price_list (owner_entity_id) WHERE kind = 'RETAIL' AND
  status = 'DRAFT';

CREATE TABLE pricing.price_list_line (
  line_id uuid PRIMARY KEY, price_list_id uuid NOT NULL REFERENCES pricing.price_list,
  sku_id uuid NOT NULL, uom_code varchar(10) NOT NULL, tier_from_qty numeric(14,3) NOT NULL DEFAULT 0,
  price numeric(14,4) NOT NULL CHECK (price >= 0), effective_from date NOT NULL, effective_to date,
  owner_entity_id uuid NOT NULL,
  EXCLUDE USING gist (price_list_id WITH =, sku_id WITH =, uom_code WITH =, tier_from_qty WITH =,
    daterange(effective_from, coalesce(effective_to,'infinity'::date), '[') WITH &&));
CREATE INDEX pll_lookup ON pricing.price_list_line (price_list_id, sku_id, uom_code, effective_from DESC);

CREATE TABLE pricing.discount_rule (
  rule_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, name text NOT NULL,
  kind text NOT NULL CHECK (kind IN
    ('TIME_LIMITED_PRICE','QUANTITY_BREAK','BILL_THRESHOLD','FREE_ITEM','EXPIRY_MARKDOWN')),
  predicate jsonb NOT NULL, benefit jsonb NOT NULL, priority smallint NOT NULL DEFAULT 100,
  advisory boolean NOT NULL DEFAULT false, adopted_from_rule_id uuid,
  valid_from date NOT NULL, valid_to date, status text NOT NULL DEFAULT 'DRAFT'
  CHECK (status IN ('DRAFT','ACTIVE','EXPIRED','WITHDRAWN')), CHECK (valid_to IS NULL OR valid_to >=
    valid_from));
CREATE INDEX rule_active ON pricing.discount_rule (owner_entity_id, status, valid_from, valid_to);

CREATE TABLE pricing.control_price (
  control_price_id uuid PRIMARY KEY, sku_id uuid, tag_code varchar(40), ceiling_price numeric(14,2) NOT NULL
  CHECK (ceiling_price > 0),
  ceiling_uom_code varchar(10) NOT NULL, effective_from date NOT NULL, effective_to date,
  gazette_reference varchar(80) NOT NULL, entered_by uuid NOT NULL, entered_at timestamptz NOT NULL DEFAULT
    now(),
  owner_entity_id uuid NOT NULL, -- always the Federation
  CHECK ((sku_id IS NULL) <> (tag_code IS NULL)),
  EXCLUDE USING gist (coalesce(sku_id,'00000000-0000-0000-0000-000000000000'::uuid) WITH =,
    coalesce(tag_code,'') WITH =,
    daterange(effective_from, coalesce(effective_to,'infinity'::date), '[') WITH &&)); --
  B-I6
GRANT INSERT ON pricing.control_price TO app_rw; GRANT UPDATE (effective_to) ON pricing.control_price TO
  app_rw; -- never edited otherwise

CREATE TABLE pricing.mrp_policy (
  policy_id uuid PRIMARY KEY, sku_id uuid, tag_code varchar(40),
  policy text NOT NULL CHECK (policy IN ('AUTO_LOWEST','BARCODE_RESOLVED','PICKER')),
  picker_gap_amount numeric(14,2), picker_gap_percent numeric(5,2), owner_entity_id uuid NOT NULL,
  CHECK ((sku_id IS NULL) <> (tag_code IS NULL)), UNIQUE
    (coalesce(sku_id,'00000000-0000-0000-0000-000000000000'::uuid), coalesce(tag_code,''),
    owner_entity_id));

CREATE TABLE pricing.adjustment_type (
  code varchar(24) PRIMARY KEY, name_en text NOT NULL, kind text NOT NULL CHECK (kind IN
    ('SPECIAL_DISCOUNT','DELIVERY_FEE')),
  permission_code varchar(64) NOT NULL, max_percent numeric(5,2), max_amount numeric(14,2),
  status text NOT NULL DEFAULT 'ACTIVE', owner_entity_id uuid NOT NULL);

CREATE TABLE pricing.label_job (job_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_id uuid NOT
  NULL,
  template text NOT NULL, sku_ids uuid[] NOT NULL, refused jsonb NOT NULL DEFAULT '{}', object_key text,
  status text NOT NULL DEFAULT 'QUEUED' CHECK (status IN ('QUEUED','RENDERED','FAILED')), created_by uuid NOT
  NULL, created_at timestamptz NOT NULL DEFAULT now());

-- RLS: template of 17A 6.3; plus PARTY read on price_list/line for the relationship's buyer (join via M1's
  relationship view);
-- control_price, advisory lists and advisory rules readable by every authenticated scope.
```

3.1 Seed data

```
# seed/m3/config.yaml    pricing.stacking_policy PRIORITY_THEN_BEST | pricing.default_mrp_policy AUTO_LOWEST |
    pricing.picker_gap {amount: 20, percent: 5}
#                        pricing.cash_rounding NEAREST_RUPEE | pricing.trade_above_mrp_review true |
    pricing.control_price_staleness_days 3
# seed/m3/adjustment-types.yaml
- {code: SPECIAL_DISC, kind: SPECIAL_DISCOUNT, permission: prc.adjust.special, max_percent: 5}
- {code: DELIVERY_FEE, kind: DELIVERY_FEE, permission: prc.adjust.delivery, max_amount: 25000}
# seed/m3/permissions.yaml (appended to the catalogue file)
- prc.pricelist.author (ENTITY) - prc.pricelist.publish (ENTITY, mfa for TRADE via handler) - prc.rule.author
  (ENTITY)
- prc.rule.activate (ENTITY) - prc.controlprice.enter (F, mfa) - prc.mrp_policy.set (ENTITY; tags F) -
  prc.adjustment.define (F)
- prc.adjust.special (F) - prc.adjust.delivery (F) - prc.label.print (ENTITY)
# seed/m3/rule-schemas/*.json one JSON schema per rule kind (predicate + benefit), used by the authoring
  validator and by the engine's loader
```

4. Package layout

```
backend/shared-engine/src/main/kotlin/lk/coopfed/noweb/engine/          -- pure Kotlin JVM, no Spring, no
  Android
  Money.kt  Quantity.kt  Tax.kt  Rounding.kt                          -- 17A skeleton, completed here
model/      RetailLine, BatchCandidate, Ceiling, Policy, Rule, Basket, LineInput, LineResult, BasketResult
PriceResolver.kt  resolveLine(input): LineResult                    -- steps 1-4 of doc 23 3.5
RuleEvaluator.kt  applyLineRules / applyBillRule                  -- step 5
BasketResolver.kt  resolve(basket, snapshot, businessDate, tenderKind): BasketResult -- steps 1-7
  orchestration
snapshot/ PricingSnapshotIndex (rules by sku/tag, ceilings by sku/tag, lines by sku/uom) built once per
  snapshot version
EngineVersion.kt      const val ENGINE_VERSION = "1.0.0"
backend/shared-engine/src/test/resources/golden/*.json                -- shared with the till (Section 9)

lk.coopfed.noweb.m3pricing/      @ApplicationModule(allowedDependencies =
  {"kernel", "m1party::api", "m1party::query", "m2catalogue::api", "m2catalogue::query", "m5inventory::query"})
api/      commands (11), events (6 families), PricingApi facade, value types
query/     PricingQueries (resolveRetailPrice, resolveBasket, resolveTradePrice, skusOnRetailList,
  ceilings, snapshot), MarkdownQueries
internal/
  list/     PriceList, Line, handlers, VersionCarryForward
  rule/     Rule, handlers, RuleSchemaValidator
  ceiling/  ControlPrice, handlers, CeilingResolver (sku + tags -> lowest in force)
  policy/   MrpPolicy, handlers, EffectivePolicy (sku beats tag; default)
  adjust/   AdjustmentType, handler
  label/    LabelJob, PrintShelfLabelsHandler, LabelRenderer (uses A4Renderer with the label template)
  authoring/ AuthoringValidator (ceilings, tiers, schema), TradeAboveMrpReview
  review/   BatchMrpReviewConsumer (batch.registered.v1), GazetteReviewConsumer (control_price.entered.v1)
  snapshot/ PricingSnapshotContributor; PublishFanOut (worker consumer of price_list.published.v1)
  queries/  PricingQueriesImpl (loads inputs, calls the engine)
web/       PriceListController, RuleController, ControlPriceController, PolicyController,
  AdjustmentController, LabelController, ReviewController
```

5. API slice: openapi/m3pricing.yaml

```
paths:
  /v1/pricing/lists:
    post: createPriceList
  /v1/pricing/lists/{listId}/versions:
    get: listPriceLists (kind, status)
    post: draftNewVersion
  /v1/pricing/lists/{listId}/lines:
    put: setLines (bulk) -> per-line outcomes {skuId, uom, tier, ok
      | ceiling: {kind, value, ref}}
  /v1/pricing/lists/{listId}/publish:
    post: publishPriceList {applyFrom} (x-mfa when kind = TRADE)
  /v1/pricing/lists/{listId}/withdraw:
    post
  /v1/pricing/rules:
    post: authorRule
  /v1/pricing/rules/{ruleId}:
    get: listRules (status, kind)
    patch: amendDraftRule
  /v1/pricing/control-prices:
    post /activate | /withdraw | /adopt
    post: enterControlPrice (prc.controlprice.enter, mfa)
  /v1/pricing/control-prices/{id}/rescind:
    get:
    post: controlPricesInForce (date)
  /v1/pricing/mrp-policies:
    put: setMrpPolicy
  /v1/pricing/adjustment-types:
    get
    post | get
  /v1/pricing/labels:
    post: printShelfLabels -> {jobId}
  /v1/pricing/resolve/retail:
    get /{jobId}
    post: resolveBasket {locationId, businessDate, tenderKind,
      lines[{skuId, uom, qty, batchId?, pickedBatchId?}]}
  /v1/pricing/resolve/trade:
    post: resolveTradePrice {relationshipId, skuId, uom, qty, date}
  /v1/pricing/authoring/check:
    post: checkCeilingsForAuthoring {entityId, skuId, uom, price}
  /v1/pricing/reviews:
    get: listReviews (kind, status)
components.schemas:
  LineResult: { skuId, uom, qty, batchId?, unitPrice, mrpApplied?, controlPriceApplied?, capReason, ruleId?,
    discountAmount,
    taxRatePercent, taxAmount, lineTotal, candidates?: [{batchId, batchNo, printedMrp, expiry,
      thumbKey}] } # candidates only for PICKER
  BasketResult: { lines: [LineResult], billRuleId?, billDiscount, subtotal, total, rounding, engineVersion,
    snapshotVersion }
# example - resolveBasket for doc 13 scenario 4 (two batches, AUTO_LOWEST, 5% rule)
{ "lines": [ { "skuId": "...", "uom": "EA", "qty": 1, "batchId": "<batch 980>", "unitPrice": 931.00,
  "mrpApplied": 980.00,
  "capReason": "MRP_LOWEST", "ruleId": "<5% rule>", "discountAmount": 49.00, "taxRatePercent":
    18.00,
  "taxAmount": 142.02, "lineTotal": 931.00 } ], "billDiscount": 0, "subtotal": 931.00, "total":
    931.00, "rounding": 0.00,
  "engineVersion": "1.0.0", "snapshotVersion": 4182 }
```

6. The shared engine (Kotlin)

Restated and formalised from doc 23 §3.5. The code below is the reference implementation the till links; the repository file is authoritative, this is what reviewers check it against.

```
// model
data class RetailLine(val skuId: Id, val uom: String, val price: Money, val tierFromQty: Qty = Qty.ZERO)
data class BatchCandidate(val batchId: Id, val batchNo: String, val printedMrp: Money?, val expiry:
  LocalDate?, val onHand: Qty)
data class Ceiling(val price: Money, val uom: String, val ref: String) // control price,
  unit-converted by the index
data class Policy(val kind: PolicyKind, val gapAmount: Money?, val gapPercent: BigDecimal?)
data class LineInput(val skuId: Id, val uom: String, val qty: Qty, val scannedBatchId: Id?, val pickedBatchId:
  Id?)
enum class CapReason { NONE, MRP_LOWEST, MRP_BARCODE, MRP_PICKED, CONTROL_PRICE }

object PriceResolver {
  fun resolveLine(i: LineInput, ix: PricingSnapshotIndex, date: LocalDate): LineResult {
    val list = ix.retailLine(i.skuId, i.uom, date) ?: return LineResult.notSellable(i, "price.no_list_line")
    val cands = ix.inStockBatches(i.skuId).filter { it.onHand > Qty.ZERO }
    val policy = ix.policyFor(i.skuId)
    val (batchTerm, batch, reasonIfBound) = when {
      !ix.sku(i.skuId).hasPrintedMrp -> Triple(null, i.scannedBatchId?.let { ix.batch(it) }, CapReason.NONE)
      policy.kind == BARCODE_RESOLVED && i.scannedBatchId != null ->
        ix.batch(i.scannedBatchId).let { Triple(it.printedMrp, it, CapReason.MRP_BARCODE) }
      policy.kind == PICKER && i.pickedBatchId != null ->
        ix.batch(i.pickedBatchId).let { Triple(it.printedMrp, it, CapReason.MRP_PICKED) }
      policy.kind == PICKER && gapExceeds(cands, policy) && i.pickedBatchId == null ->
        return LineResult.needsPick(i, cands) // till shows the picker
      else -> cands.minByOrNull { it.printedMrp ?: Money.MAX }.let { Triple(it?.printedMrp, it,
        CapReason.MRP_LOWEST) }
    }
    val control = ix.ceilingFor(i.skuId, i.uom, date) // lowest of sku- and
      tag-scoped
    var unit = list.price; var reason = CapReason.NONE
    if (batchTerm != null && batchTerm < unit) { unit = batchTerm; reason = reasonIfBound }
    if (control != null && control.price < unit) { unit = control.price; reason = CapReason.CONTROL_PRICE }
    return LineResult.base(i, batch, unit, mrpApplied = batchTerm, controlApplied = control?.price, capReason
      = reason)
  }
}
```

```

    }
}

object RuleEvaluator {
    fun applyLineRules(r: LineResult, ix: PricingSnapshotIndex, date: LocalDate, policy: StackingPolicy):
        LineResult {
        val applicable = ix.lineRulesFor(r.skuId, date).filter { it.matches(r) } // kind-specific
        predicate
        val chosen = when (policy) {
            PRIORITY_THEN_BEST -> applicable.sortedWith(compareBy({ it.priority }, { -benefitOf(it, r)
                })).firstOrNull()
            BEST_ONLY -> applicable.maxByOrNull { benefitOf(it, r) } }
        return chosen?.let { r.withBenefit(it.id, benefitOf(it, r), it.kind == EXPIRY_MARKDOWN) } ?: r
    }
    fun applyBillRule(lines: List<LineResult>, ix: PricingSnapshotIndex, date: LocalDate): BillBenefit? = ...
}

object BasketResolver {
    fun resolve(b: Basket, ix: PricingSnapshotIndex, date: LocalDate, tender: TenderKind): BasketResult {
        val lines = b.lines.map { PriceResolver.resolveLine(it, ix, date) }
        .map { if (it.sellable) RuleEvaluator.applyLineRules(it, ix, date, ix.stackingPolicy) else it }
        .map { it.recheckCeiling(ix, date) } // step 6: never
        above control
        val bill = RuleEvaluator.applyBillRule(lines, ix, date)
        val taxed = Tax.decompose(lines, bill, ix) // inclusive -> tax
        amounts, apportion bill benefit
        val total = taxed.sumOf { it.lineTotal } - (bill?.amount ?: Money.ZERO)
        val rounding = if (tender == CASH) Rounding.toRupee(total) else Money.ZERO
        return BasketResult(taxed, bill, total + rounding, rounding, ENGINE_VERSION, ix.snapshotVersion)
    }
}

```

Aspect	Design
Index	PricingSnapshotIndex is built once per snapshot version from the snapshot rows: maps sku → retail lines by uom and tier, sku/tag → active rules, sku/tag → ceilings (unit-converted using M2 conversions), sku → policy, sku → in-stock batches (from the lot rows). Building it is O(rows); resolving is O(lines × rules-for-sku).
Determinism	No clocks, no randomness, no I/O; the business date is an input; money arithmetic uses the library's fixed rounding modes; the same inputs give the same bytes on JVM and Android (a golden fixture asserts the serialised BasketResult).
Central use	PricingQueriesImpl.resolveBasket loads the same inputs from tables into the same index type and calls the same function; ingestion re-validation rebuilds the index for the snapshot version the till reported (kernel keeps snapshot versions for the change-log window).

7. Handler specifications, consumers, snapshot

Command	Guards (in order)	Mutation	Audit	Events
CreatePriceList	owner; kind; RETAIL: no DRAFT exists (unique index backstop); ADVISORY/TRADE-federation: caller F	insert DRAFT	PRICELIST_DRAFTED	–
DraftNewVersion	source PUBLISHED or SUPERSEDED; owner	new DRAFT with lines carried forward (effective_from = null until publish)	PRICELIST_DRAFTED	–
SetLines	list DRAFT; owner; per line: sku active, uom valid (M2), tiers ascending, price ≥ 0; AuthoringValidator.ceiling (RETAIL: min in-stock batch MRP at owner's locations via M5 query, control price; TRADE: review-only above lowest MRP)	upsert draft lines; return per-line outcomes	–	–
PublishPriceList	DRAFT; all lines valid at publish time (re-run validator); apply_from ≥ today; TRADE: MFA	set effective_from = apply_from on lines; close previous version lines at apply_from – 1; status PUBLISHED, previous SUPERSEDED	PRICELIST_PUBLISHED	price_list.published.v1 (changed sku ids)
AuthorRule / AmendDraftRule	owner; RuleSchemaValidator for the kind; scope sku/tag exists; EXPIRY_MARKDOWN requires expiry-tracked scope (M2 flag; for a tag, every SKU under it or the validator lists offenders); advisory only by F	insert / update DRAFT	RULE_AUTHORED	–
ActivateRule	DRAFT; valid_from ≥ today; FIXED_PRICE benefit under ceilings for its scope today; benefit within [0, 100%]	ACTIVE; change-log rows for the owner's locations (apply_from = valid_from)	RULE_ACTIVATED	discount_rule.activated.v1
WithdrawRule	ACTIVE; reason	WITHDRAWN; change-log urgent	RULE_WITHDRAWN	discount_rule.withdrawn.v1
AdoptAdvisoryRule	source advisory ACTIVE; caller MPCs	copy as DRAFT with adopted_from	RULE_AUTHORED	–
EnterControlPrice	caller F; MFA; scope exists (sku or governed tag); ceiling > 0; gazette ref; overlap check (exclusion backstop)	insert; close previous same-scope row at effective_from – 1; change-log urgent to all locations (worker fan-out)	CONTROL_PRICE_ENTERED	control_price.entered.v1
RescindControlPrice	IN_FORCE; reason and reference	set effective_to; change-log urgent	CONTROL_PRICE_RESCINDED	control_price.rescinded.v1
SetMrpPolicy	owner (F for tag scope); gap fields only for PICKER	upsert; publish	MRP_POLICY_SET	mrp_policy.changed.v1 (M2 updates sku.multi_mrp_policy)
DefineAdjustmentType	caller F; bounds present	insert	ADJUSTMENT_TYPE_DEFINED	adjustment_type.defined.v1
PrintShelfLabels	owner; location belongs to owner; SKUs on the retail list	label_job QUEUED; job renders; per SKU: refuse when list price > ceiling in force on the print date; refused[] filled	LABEL_JOB_CREATED	label_job.rendered.v1

7.1 Consumers

```

@EventConsumer(types={"batch.registered.v1"}, consumer="m3.batch-mrp-review")
  // for each MPCs whose PUBLISHED retail line for the sku exceeds the batch's printed MRP (and that holds or
  // may receive the batch:
  // in phase 1: every MPCs with a retail line for the sku) -> pricing.review_raised.v1 BATCH_MRP_BELOW_LIST +
  // exception-queue audit REVIEW
@EventConsumer(types={"control_price.entered.v1"}, consumer="m3.gazette-review")
  // for every PUBLISHED retail list line (and TRADE line) above the new ceiling ->
  LIST_ABOVE_CEILING_AT_GAZETTE review per owner
@EventConsumer(types={"price_list.published.v1"}, consumer="m3.publish-fanout") // worker

```

```
// RETAIL: for each location of the owner: SnapshotChangeLog.record(location, "price_list_line", lineId,
    UPSERT, applyFrom, false)
// for changed skus; TOMBSTONE for removed lines
@ScheduledJob(name="rule-expiry", cron="0 5 0 * * *") // ACTIVE rules past valid_to -> EXPIRED (no snapshot
    row needed: tills evaluate validity)
```

7.2 Snapshot contributor

```
tables(): ["price_list_line", "discount_rule", "control_price", "mrp_policy"]
rowsFor(location, table, ids):
    price_list_line -> lines of the owner MPC's PUBLISHED (and SUPERSEDED-but-still-effective) retail list for
        the assortment; applyFrom = effective_from
    discount_rule -> ACTIVE (and DRAFT->ACTIVE scheduled) rules of the owner MPC's; applyFrom = valid_from
    control_price -> all rows in force or future; applyFrom = effective_from; urgent on entry/rescind
    mrp_policy -> effective policies for the assortment's skus (sku rows plus tag rows for their tags)
```

8. Screens

Offer notice (rule editor)	Shelf price list (retail)
+-----+	
Kind: [Time-limited price v]	Version 12 DRAFT (from v11) apply from
[2026-10-05]	search [____] tag [any v] show [all issues]
Applies to: (o) item [Milk powder 400g]	SKU name unit advisory price
	7Q2K9 Milk powder EA 1,050 1,040
() tag [____]	3F8A1 Red rice 1kg KG - 240 !
ceiling	! = above control price (gazette 2492/29) -
Benefit: () fixed price [Rs ____]	[Print labels...] [Publish...]
ok	
(o) percent off [5] %	
220	
Valid: [2026-10-01] to [2026-10-31] Priority [100]	
fix to publish	
Preview on sample basket: [Try] -> 931.00 shown	
(disabled: 1 issue)	
[Save draft] [Activate] -> StepUp if F	
+-----+	

Screen	Components	Message ids (m3.)	Notes
Wholesale price list	DocumentHeader, versions, tier table, review flags	list.*	MFA on publish
Shelf price list	as sketched; ceiling status per line from the validator	list.*	Advisory column read-only
Offer notice	kind-driven form, preview (calls resolveBasket), ApprovalBar	rule.*	No free-text logic anywhere
Gazette	form with gazette reference, ReasonCapture, StepUpDialog, affected-lists count	ceiling.*	F only
MRP policy	table by SKU/tag	policy.*	–
Pricing reviews	ExceptionQueue filtered to pricing kinds with deep link to the list line	review.*	–
Markdown queue	list from M5 expiry data joined with the applicable rule	markdown.*	Web and till (M6 renders)

9. Tests

Layer	Cases	Target
Engine unit	Each step of doc 23 §3.5; every rule kind and benefit; stacking policies; tax decomposition incl. bill apportionment; rounding (.49/.50/.51); ceiling unit conversion (per kg ceiling vs 400 g pack)	Every branch
Engine property-based	Random snapshots and baskets: total $\leq$ sum of list prices; every line $\leq$ control term; capReason consistent; idempotent re-resolution; serialised result identical on JVM and Android (parity harness in till/app androidTest)	10,000 cases; parity 100%
Golden baskets	golden/* .json: doc 13 scenario 4 plus twenty fixtures (single batch, two batches, picker, gazette below list, expiry markdown, free item out of stock, bill threshold with tag restriction, cash vs card rounding, no list line, has_printed_mrp false)	All green on both platforms
Contract	Every operation; setLines per-line outcomes carry the binding ceiling; resolveBasket example above reproduces to the cent	Every operation
State machine	Lists, rules, control prices; urgent flags on withdraw/enter/rescind	All transitions
Consumers	BATCH_MRP_BELOW_LIST raised only when below; gazette review fan-out; publish fan-out produces exactly the expected change-log rows per location	Three cases
Scenario	Flows 6.1–6.7 of doc 23 end to end (M5 stub for in-stock batches; M4 stub for adjustments)	Seven flows
Performance	30-line basket, 200 rules: under 5 ms on the reference terminal, under 2 ms on JVM; publish fan-out for 6,000 SKUs $\times$ 12 locations under 30 s	Two targets
Web	Playwright: draft list with an over-ceiling line $\rightarrow$ publish disabled $\rightarrow$ fix $\rightarrow$ publish; author and activate an expiry rule; enter a control price with step-up	Three flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M3-01	Engine model and index; Money/Quantity/Tax/Rounding completion in shared-engine	1.5	Unit tests green
M3-02	PriceResolver, RuleEvaluator, BasketResolver; golden baskets; property tests; parity harness with the till track	4	Golden and parity green on JVM and Android
M3-03	Migration V0001 with RLS; seeds; rule JSON schemas	1	Applies; schemas validate fixtures
M3-04	Price lists: create, versions, set lines with AuthoringValidator, publish with carry-forward and closure; PublishFanOut	3	Flow 6.1; fan-out test
M3-05	Rules: author, amend, activate, withdraw, adopt; schema validator; expiry job	2	Flow 6.3
M3-06	Control prices: enter, rescind, CeilingResolver with unit conversion; gazette review consumer; urgent change-log	2	Flow 6.4
M3-07	MRP policy, EffectivePolicy, M2 denormalisation event; adjustment types; ResolveTradePrice	1.5	Flow 6.7; M2 sku field updated
M3-08	Batch MRP review consumer; reviews query; queries incl. resolveBasket/resolveRetailPrice at central	1.5	Flow 6.5, 6.6
M3-09	Snapshot contributor; label job with ceiling refusal	1.5	Snapshot tests; refused labels listed
M3-10	Web screens (seven) and Playwright flows	3	Three flows green
M3-11	Contract freeze: m3pricing.yaml v1; engine 1.0.0 tagged; api/query review with M4 and M6; doc 33 slice	0.5	M4 and M6 acknowledge

Total about 21.5 developer-days. The engine (M3-01, M3-02) is the first deliverable and is what the till track integrates; tag engine 1.0.0 before the module contract freeze so M6's spike work links a stable artefact.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Stacking policy (doc 23 DR-1)	PRIORITY_THEN_BEST	Federation admins	Config value; engine already supports BEST_ONLY

Point	Assume	Decider	If different
SKU vs tag ceiling (DR-2)	Lower applies	Federation legal	CeilingResolver picks by specificity instead; one-line change, golden fixture changes
Trade above MRP (DR-3)	Review, not block	Federation	Turn the review into an authoring refusal
Picker gap (DR-4)	Rs 20 or 5%	Federation + usability round	Config values
Advisory adoption (DR-5)	By copy	Federation	Remove AdoptAdvisoryRule
New tables (DR-6) and P3 slot (DR-7)	Adopted	Architecture review	–
M5 InStockBatches query	Stub returning fixture batches until doc 25	M5 developer	Swap the stub
Ceiling unit conversion for weighed goods	Per-kg ceilings convert through M2 conversions; loose goods priced per kg compare directly	M2 and M3 developers	–

End of document. 23A v0.1; the shared-engine README and golden fixtures, and m3pricing/README.md, become the living version once merged.